AIMLCZG523 MLOps Assignment 01

End-to-End Reproducible Heart Disease Prediction Pipeline

Student name: TO BE FILLED

Roll number: TO BE FILLED

Email: TO BE FILLED

Repository: https://github.com/Sainagasashank/heart-disease-mlops-pipeline

This report is generated from the current model outputs and evidence files.

1. Setup and installation

Create a clean Python 3.12 virtual environment, then run pip install -r requirements-dev.txt.

Primary commands: make data, make train, make test, make serve.

Container workflow: make docker-build then make docker-run. Kubernetes and Helm commands are provided for an owner-operated cluster.

Monitoring: make monitoring-up; use make traffic to generate requests.

2. EDA and preprocessing

The Cleveland dataset has 303 records. Raw '?' values are converted to missing values.

Numerical variables use median imputation and standard scaling; categorical variables use most-frequent imputation and one-hot encoding. The same Pipeline is saved and served, avoiding training-serving skew.

3. Model selection and nested tuning

Model	Grid Size	Best Inner-CV AUC	Best Params
0.9109	{\classifier__C\": 0.1	\"classifier__class_weight\": null	\"classifier__penalty\": \"l2\"}"
0.9028	{\classifier__max_depth\": 3	\"classifier__min_samples_leaf\": 2	\"classifier__n_estimators\": 200}"
0.8874	{\classifier__learning_rate\": 0.03	\"classifier__max_depth\": 2	\"classifier__n_estimators\": 100}"

Model	CV Accuracy	CV ROC-AUC	Test Accuracy	Test Precision	Test Recall	Test F1	Test ROC-AUC
LogisticRegression	0.8387	0.9025	0.8852	0.8387	0.9286	0.8814	0.9665
RandomForest	0.8054	0.8962	0.8852	0.8182	0.9643	0.8852	0.9643
GradientBoosting	0.7893	0.8744	0.8852	0.8387	0.9286	0.8814	0.9513

Three candidates are tuned with GridSearchCV on an 80% training split. A 3-fold inner CV selects parameters within each outer fold; five outer folds estimate generalization. The held-out 20% test split is evaluated once.

Champion: **LogisticRegression**. Selection criterion: Highest nested-CV ROC-AUC; tie-breaker nested-CV accuracy. Held-out test metrics are not used for model selection.

4. Experiment tracking

MLflow records baseline and best parameters, inner-CV and test metrics, figures, reports, tuning result CSVs and the input-example model artifact.

Run python scripts/relocate_mlflow.py before make mlflow-ui after moving the project. The script makes artifact locations portable.

5. Quality assurance and CI/CD

Pytest covers cleaning, preprocessing, grid validity, metrics, model inference, API validation/determinism and Prometheus metrics. Flake8 is enforced without exit-zero.

GitHub Actions and Jenkins use Python 3.12, run lint/test/coverage, train artifacts, and use a health polling loop before the container prediction smoke test.

6. Serving and deployment

FastAPI exposes /health, /predict, /metrics and /docs. Input contracts reject invalid payloads. The Dockerfile uses python:3.12-slim, pinned requirements, a non-root user and curl health check.

Kubernetes manifests and the Helm chart are supplied. Their execution is owner-operated and should be evidenced on the target cluster.

7. Monitoring and logging

The compose stack runs API, Prometheus and Grafana. Prometheus scrapes api:8000 in compose; prometheus.local.yml is for a host-running API. Grafana provisioning supplies a default Prometheus datasource.

The API emits REQUEST_LOG and PREDICTION_RESULT lines, exposes request metrics, a prediction-outcome counter, and an inference latency histogram.

8. Evidence and limitations

Embedded screenshots are used only when the corresponding captured PNG exists. Docker, Kubernetes, GitHub Actions and Grafana execution are procedures until run by the owner.

No claim of image size, cluster deployment, or container endpoint success is made without a captured local run.

9. Repository link and conclusion

Repository URL: https://github.com/Sainagasashank/heart-disease-mlops-pipeline

The project provides reproducible specifications, nested-CV selection, portable experiment metadata, testing, CI/CD definitions, container/deployment manifests and monitoring configuration. Owner tasks: fill identity/repository details, execute external platforms, record video and upload the repository.